Table of Content

Table 1: Content of the report

Objectives:

1. To design a digital logic circuit implementing a Boolean function derived from the digits of my roll number (2009047).
2. To implement the design using different Verilog modeling styles: Gate-level, Dataflow, Behavioral, and Switch-level (CMOS).
3. To develop corresponding testbench programs for functional verification.
4. To analyze simulation waveforms and verify the correctness of all models.

Introduction:

Verilog is a Hardware Description Language (HDL) used to model electronic systems, allowing designers to describe the behavior and structure of digital circuits at multiple levels of abstraction. In this project, we design a combinational circuit based on a unique set of minterms derived from a specific ID (Roll Number) and explore four specific modeling styles. These include Gate Level, which describes the circuit using logic primitives (AND, OR, NOT); Dataflow, which utilizes boolean equations and continuous assignments; Behavioral, which describes the algorithm or logic flow using procedural blocks; and Switch Level, which employs transistors (NMOS/PMOS) to represent the physical CMOS structure.

Theory:

- **SOP (Sum of Products):** A Boolean function is expressed as a sum (OR) of product (AND) terms. Each product term represents a "minterm" for which the function evaluates to 1.
- **K-Map (Karnaugh Map):** A graphical tool used to simplify Boolean algebra expressions. It arranges truth table values in a grid where adjacent cells differ by only one bit (Gray code), allowing for the grouping of 1s to eliminate redundant variables.
- **Verilog Modeling Styles:** Verilog HDL allows a design to be described at different levels of abstraction. This project utilizes four key styles:
    - **Gate-Level Modeling:** This is a structural description of the circuit, analogous to a schematic. It involves instantiating predefined logic

gate primitives (e.g., and, or, not) and defining the wires that connect the

- **Dataflow Modeling:** This style describes the circuit in terms of how data flows through it. It is primarily based on continuous assignments (using the assign keyword) and Boolean equations, making it a concise way to represent combinational logic.
- **Behavioral Modeling:** This is the highest level of abstraction used here. It describes the circuit's function algorithmically using procedural blocks (like always @(*)). It focuses on *what* the circuit does (its behavior) rather than *how* it is built from gates.
- **Switch-Level Modeling:** This is the lowest level of abstraction, describing the circuit in terms of its transistor connections. By using pmos and nmos primitives, it models the actual CMOS (Complementary Metal-Oxide-Semiconductor) implementation, providing insight into the circuit's physical structure.

Problem Satement:

Design a digital circuit that implements a Boolean function, where the sum of products (SOP) terms corresponds to each digit of your roll number.

Given expression:

$F(A,B,C,D) = \sum (2,0,0,9,0,4,7)$

Solution:

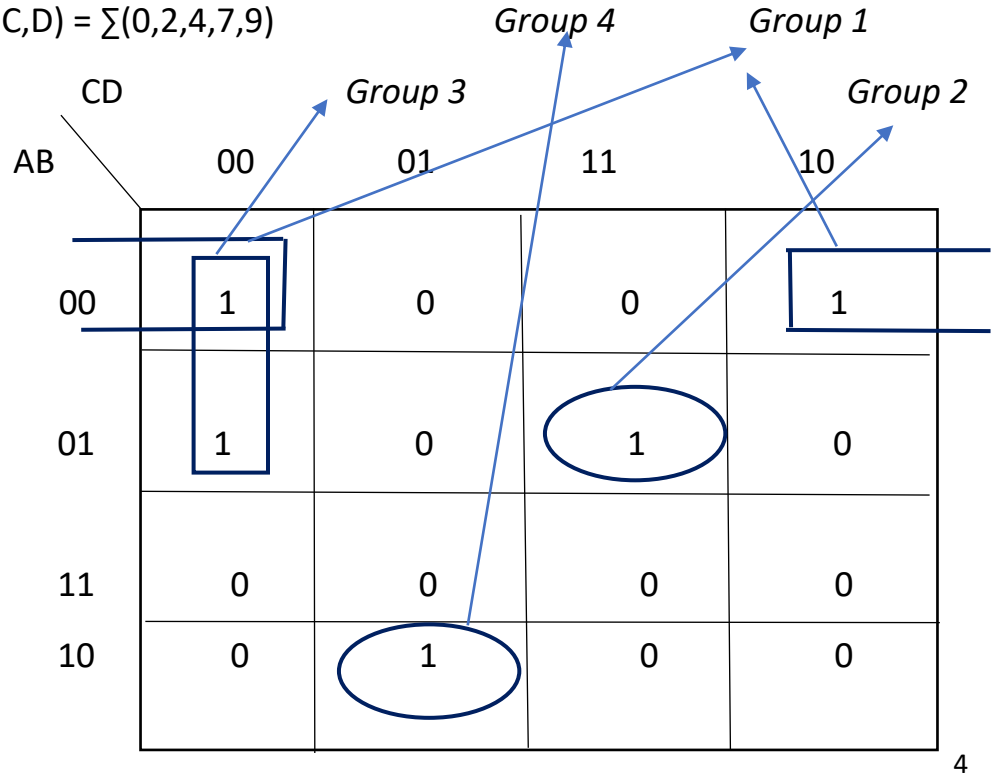
Here the function is presented in the mintems. So it will be a 4-variable (input) k-map. We can easily make the truth table as the position given will be 1. Other position will hold 0.

Table-01: truth table for the given expression

A	B	C	D	F
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Simplification of the expression using K-map:

F(A,B,C,D) = ∑(0,2,4,7,9)



Group 1:

Simplified expression: A’B’D’

Group 2:

Simplified Expression: A’BCD

Group 3:

Simplified expression: A’C’D’

Group 4:

Simplified Expression: AB’C’D

Final Expression: A’B’D’ + A’BCD + A’C’D’ + AB’C’D

Designed Circuit Diagram:

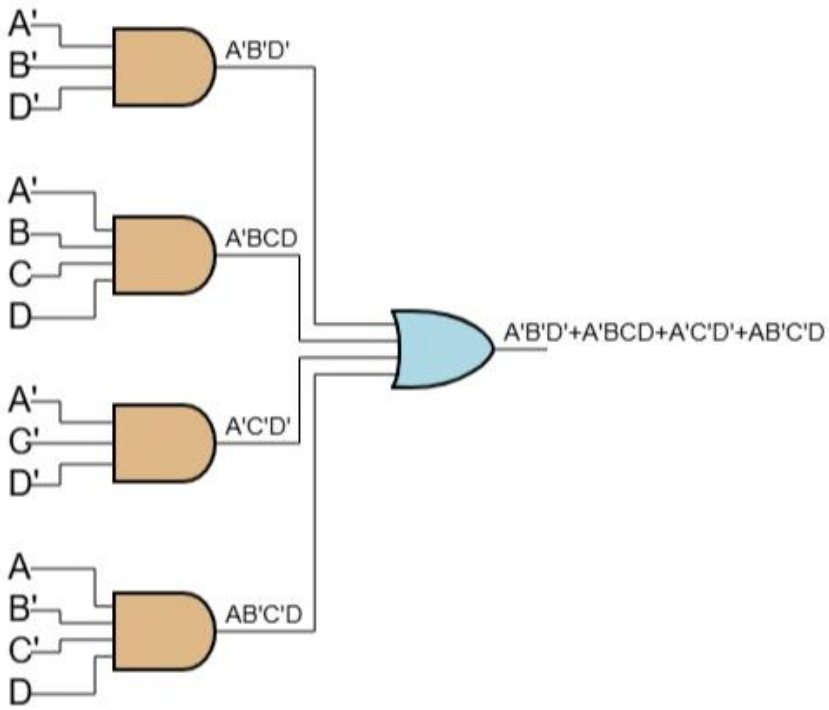


Fig 1: Designed Circuit Diagram

Program for Gate level, Data flow level, behavioral level, Switch level modeling:

Code 1: Code for Gate-level modeling

```
module roll ( s, p1, p2, p3, p4,a, b, c, d);
   input a, b, c, d;
   output s, p1, p2, p3, p4;

   wire a_n, b_n, c_n, d_n;

   not nA(a_n, a);
   not nB(b_n, b);
   not nC(c_n, c);
   not nD(d_n, d);

   and g1(p1, a_n, c_n, d_n);
   and g2(p2, a_n, b_n, d_n);
   and g3(p3, a, b_n, c_n, d);
   and g4(p4, a_n, b, c, d);

   or g_final(s, p1, p2, p3, p4);
endmodule
```

Code 2: Code for Data Flow level modeling

```
module roll_dataflow (s, p1, p2, p3, p4,a, b, c, d);
   input a, b, c, d;
   output s, p1, p2, p3, p4;
```

```
    assign p1 = (~a) & (~c) & (~d);
    assign p2 = (~a) & (~b) & (~d);
    assign p3 = (a) & (~b) & (~c) & (d);
    assign p4 = (~a) & (b) & (c) & (d);
    assign s = p1 | p2 | p3 | p4;
endmodule
```

Code 3: Code for Behavioral level modeling:

```
module roll_behavioral (s, p1, p2, p3, p4,a, b, c, d);
    input a, b, c, d;
    output reg s, p1, p2, p3, p4;
    always @(*) begin
        p1 = (~a) & (~c) & (~d);
        p2 = (~a) & (~b) & (~d);
        p3 = (a) & (~b) & (~c) & (d);
        p4 = (~a) & (b) & (c) & (d);
        s = p1 | p2 | p3 | p4;
    end
endmodule
```

Code 4: Code for Switch level modeling:

```
module cmos_roll_switch_level(f,a,b,c,d);
 input a,b,c,d;
 output f;
```

```
// --- Power Supply Definitions ---
  supply1 vdd;
  supply0 gnd;
// Wires for internal nodes in the Pull-Up Network (PMOS)
wire   f1a,f1c,f2a,f2b,f3an,f3b,f3c, f4a,f4b,f4c;
// Wires for internal nodes in the Pull-Down Network (NMOS)
  wire an,bn,cn,dn;
// Inverter for A -> an (~a)
  pmos pa (an,vdd,a);
  nmos na (an,gnd,a);
  // Inverter for AB -> bn (~b)
  pmos pb (bn,vdd,b);
  nmos nb (bn,gnd,b);
  // Inverter for C -> cn (~c)
  pmos pc (cn,vdd,c);     //pmos(drain,source,gate)
  nmos nc (cn,gnd,c);
  // Inverter for D -> dn (~d)
  pmos pd (dn,vdd,d);
  nmos nd (dn,gnd,d);

  // Pull-Up Network (PUN)
  // Branch 1: Inputs a, c, d in series
  pmos F1a (f1a,vdd,a);
  pmos F1c (f1c,f1a,c);
  pmos Fld (f,f1c,d);
// Branch 2: Inputs a, b, d in series
  pmos F2a (f2a,vdd,a);
```

```
pmos F2b (f2b,f2a,b);
 pmos F2d (f,f2b,d);
// Branch 3: Inputs an, b, c, dn in series
pmos F3a (f3a,vdd,an);    //pmos(drain,source,gate)
pmos F3b (f3b,f3a,b);
pmos F3c (f3c,f3b,c);
pmos F3d (f,f3c,dn);
// Branch 4: Inputs a, bn, cn, dn in series
pmos F4a (f4a,vdd,a);
pmos F4b (f4b,f4a,bn);
pmos F4c (f4c,f4b,cn);
pmos F4d (f,f4c,dn);

// 3. Pull-Down Network (PDN)
wire n1,n2,n3;
// Block 1 (Closest to Output): Parallel inputs a, c, d
nmos N1a (f,n1,a);    //nmos(drain,source,gate)
nmos N1c (f,n1,c);
nmos N1d (f,n1,d);

 // Block 2: Parallel inputs a, b, d
nmos N2a (n1,n2,a);
nmos N2b (n1,n2,b);
nmos N2d (n1,n2,d);
```

```
    // Block 3: Parallel inputs an, b, c, dn
      nmos N3a (n2,n3,an);
      nmos N3b (n2,n3,b);
      nmos N3c (n2,n3,c);
      nmos N3d (n2,n3,dn);
    // Block 4 (Closest to GND): Parallel inputs a, bn, cn, dn
      nmos N4a (n3,gnd,a);
      nmos N4b (n3,gnd,bn);
      nmos N4c (n3,gnd,cn);
      nmos N4d (n3,gnd,dn);
    endmodule
```

Test Bench Code(switch level):

```
    module tb_cmos_roll_switch;
       // Inputs to the design
       reg a, b, c, d;
       // Output from the design
       wire f;
       // Instantiate the Design Under Test (DUT)
       cmos_roll_switch_level dut (.f(f),.a(a), .b(b),.c(c),.d(d));
      initial begin
          // Setup waveform dumping
          $dumpfile("waveform.vcd");
          $dumpvars(1, tb_cmos_roll_switch); // Monitor all signals in this testbench
          // Print header for the console output
          $display("Time | a b c d | f");
```

```
        // Monitor changes to the signals and print them
        $monitor("%4dns | %b %b %b %b | %b",
                $time, a, b, c, d, f);
        // --- Test Vector Stimulus ---
        // Apply all 16 possible input combinations
        a = 0; b=0; c=0; d=0;
        #10 a = 0; b=0; c = 0; d=1;
        #10 a = 0; b=0; c=1; d=0;
        #10 a = 0; b=0; c=1; d=1;
        #10 a = 0; b=1; c=0; d=0;
        #10 a = 0; b=1; c=0; d=1;
        #10 a = 0; b=1; c=1; d=0;
        #10 a = 0; b=1; c=1; d=1;
        #10 a = 1; b=0; c=0; d=0;
        #10 a = 1; b=0; c=0; d=1;
        #10 a = 1; b=0; c=1; d=0;
        #10 a = 1; b=0; c=1; d=1;
        #10 a = 1; b=1; c=0; d=0;
        #10 a = 1; b=1; c=0; d=1;
        #10 a = 1; b=1; c=1; d=0;
        #10 a = 1; b=1; c=1; d=1;
        // End simulation after a final delay
        #10 $finish;
    end
endmodule
```

Result Analysis:

- From Fig 2 we found that the output epwaveform found in Verilog in case of gate level modeling , shows that the output is high when the input is ‘0000’, ‘0010’, ‘0100’, ‘0111’, ‘1001’ . which is satisfied by the truth table table:1

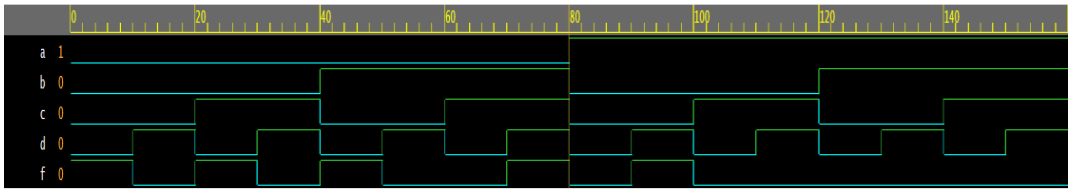


Fig 2: Output epwaveform of gate level modeling

- Fig 3 refers to that the output epwaveform found in Verilog in case of Data flow modeling , shows that the output is high when the input is ‘0000’, ‘0010’, ‘0100’, ‘0111’, ‘1001’ . which is satisfied by the truth table table:1

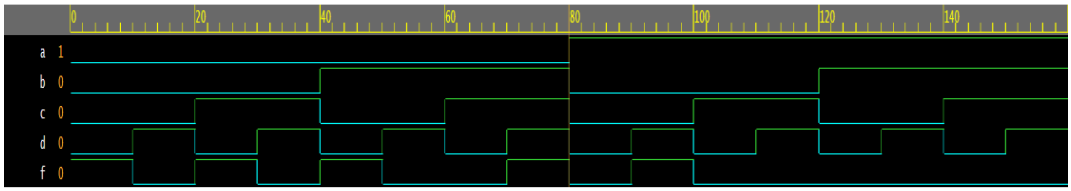


Fig 3: Output epwaveform of Data flow modeling

- Fig 4 represents that the output epwaveform found in Verilog in case of Behavioral level modeling , shows that the output is high when the input is ‘0000’, ‘0010’, ‘0100’, ‘0111’, ‘1001’ . which is satisfied by the truth table table:1

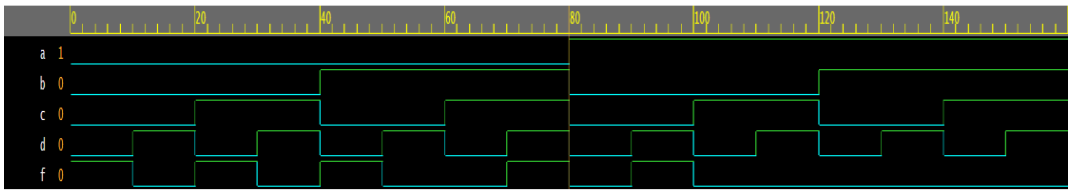


Fig 4: Output epwaveform of Behavioral level modeling

- From Fig 5, it can be that the output epwaveform found in Verilog in case of gate level modeling , shows that the output is high when the input is ‘0000’, ‘0010’, ‘0100’, ‘0111’, ‘1001’ . which is satisfied by the truth table [Table:1]

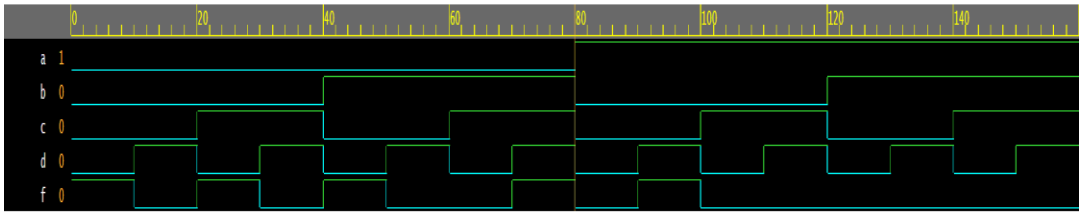


Fig 5: Output epwaveform of Behavioral level modeling

Discussion:

This project was carried out to design a digital circuit whose logic is based on the digits of the roll number, resulting in the Boolean function that produces output 1 for minterms 0, 2, 4, 7 and 9. The main aim was to implement this circuit using four different abstraction levels in Verilog and observe how each modeling style behaves during simulation. Through the work, it became clear that each modeling style offers a different way of thinking about the same circuit. At the gate level, the design had to be built using actual logic gates, which showed how the SOP expression is realized using AND, OR, and NOT gates. This gave a very structural view of the circuit. In the data flow level, the design became simpler because the logic was written directly as a Boolean equation using continuous assignments. This made the code shorter and easier to understand, and the output updated immediately whenever the inputs changed. The behavioral model was even more abstract, using procedural statements like always blocks. This style focused only on the logic relationship and did not reflect physical hardware, giving ideal and very clean waveforms. The switch-level model was the most low-level representation, where the circuit was built using NMOS and PMOS transistors. This model helped in visualizing how the hardware actually conducts or blocks signals depending on the transistor states. Even though it is more complex, the switch-level simulation still matched the expected logic perfectly. Across all models, the output F consistently became 1 only for the correct input combinations (0, 2, 4, 7 and 9), proving that every model accurately represented the truth table given in Table-01. Overall, the project showed how the same logic function behaves consistently across different levels of

abstraction, while each modeling style provides a unique perspective on digital circuit design. It also helped in understanding the strengths and uses of each modeling style in Verilog.

Conclusion:

In this project, a digital circuit was designed in Verilog HDL based on the SOP terms derived from the digits of the roll number. At the beginning, the basic concepts of SOP, POS, and K-map simplification were reviewed to understand how Boolean expressions are formed and minimized. After obtaining the truth table from the given minterms, the expression was simplified using a K map, and a corresponding logic circuit was drawn to visualize the hardware implementation. Using simplified expression, the circuit was then implemented in four Verilog modeling styles: gate level, data flow level, behavioral level, and switch level. Separate testbench programs were written to verify the correctness of each model using all possible input combinations from the truth table. All codes were simulated using an online Verilog simulator (EDA Playground), and the generated timing diagrams and console outputs showed that every implementation produced the correct results. For all models, the output matched the expected truth table, confirming that the design was functionally accurate across all abstraction levels. This successful verification demonstrates a clear understanding of how digital circuits can be described structurally, behaviorally, and at the transistor level in Verilog. Overall, this project provided valuable hands-on experience in digital system design and strengthened the understanding of Verilog as a powerful hardware description language. Since Verilog is widely used in microcontroller, processor, and VLSI design, the skills gained through this project will be highly beneficial for future work in hardware development and embedded system design.

Reference:

- Digital Logic And Computer Design. - M.Morris Mano.
- https://www.allaboutcircuits.com/
- Lab manual of VLSI design and Nanotechnology Labboratory